RAPPORT DE PROJET ACADÉMIQUE

Plateforme Cloud Décentralisée Pilotée par Registre Blockchain (ChainOps)

Orchestration d'Infrastructure Immuable sur Cluster Physique Raspberry Pi avec K3s, Consensus Raft et immudb

Module : BlockChain & Smart Contract

Année Universitaire 2026/2027

Réalisé par :

SALIOUI Ilyes **SBAI Ahmed Taha**

SEBAI EL FASSI Abdelmonim **ASLOUNE Yahya**

Sous la direction et l'encadrement de :

Mme Meryeme AYACHE

Professeur à l'Institut National des Postes et Télécommunications (INPT)

Filière : Systèmes Embarqués et Services Numériques (SESNUM – INE3)

Institut National des Postes et Télécommunications – Rabat

« L'infrastructure cloud moderne ne doit plus reposer sur la confiance aveugle envers un administrateur central, mais sur des preuves cryptographiques immuables et un consensus décentralisé tolérant aux pannes. »

Table des matières

Résumé Exécutif **1**

Abstract **2**

1 Introduction, Contexte et Problématique **3**
1.1 Contexte Technologique : Les Limites du Cloud Centralisé 3
1.2 Le Paradigme de l'« Infrastructure as a Ledger » 3
1.3 Objectifs du Projet et Cahier des Charges Académique 4

2 Architecture Matérielle et Topologie Réseau Zero-Trust **5**
2.1 Spécifications du Cluster Physique Raspberry Pi 4 5
2.2 Topologie Réseau Globale et Routage Zero Trust 5
2.3 Résolution du Défi d'Ingress : Du Proxy HTTPS Layer 7 au Tunnel TCP Layer 4 6

3 Le Moteur Blockchain en 5 Étapes (Fonctionnement Interne) **8**
3.1 Étape 1 : Cryptographie Côté Client et Signatures Asymétriques 8
3.2 Étape 2 : Le Smart Contract Gatekeeper en Go 9
3.3 Étape 3 : Moteur de Consensus Distribué Raft 9
3.4 Étape 4 : Registre Immuable immudb et Arbre de Merkle 10
3.5 Étape 5 : Ordonnancement Physique K3s via client-go 10

4 Sécurité Cryptographique et Détection d'Altération **12**
4.1 Formulation Mathématique du Chaînage d'État 12
4.2 Vérification d'Audit de Bout en Bout 12
4.3 Démonstration de Détection d'Attaque Malveillante 13

5 Résultats Expérimentaux et Visite Visuelle **14**
5.1 Authentification Cryptographique et Interface de Connexion 14
5.2 Cycle de Déploiement en 5 Étapes et Quotas 14
5.3 Exécution en Production et Télémétrie Matérielle 15
5.4 Télémétrie du Cluster et Consensus Raft 16
5.5 Explorateur de Registre Blockchain et Racine de Merkle 17
5.6 Vérification du Smart Contract : Protection Anti-OOM 17
5.7 Validation sur Console Physique Master 18
5.8 Infrastructure Cloudflare Zero Trust et Environnement Local 19

6 Spécification des APIs et Guide d'Exploitation **20**
6.1 Spécification des Endpoints REST du Démon Go 20
6.2 Automatisation du Déploiement et Exploitation 20
6.2.1 Déploiement du Nœud Maître (Master Pi) 21
6.2.2 Déploiement des Nœuds Ouvriers (Worker Pis) 21
6.2.3 Purge des Nœuds Inactifs 21

7 Conclusion et Perspectives d'Avenir **22**
7.1 Bilan des Réalisations Techniques et Académiques 22
7.2 Perspectives d'Avenir et Axes de Recherche 22

Table des figures

1 Architecture réseau Zero Trust avec routage Layer 4 TCP public 6
2 Le pipeline cryptographique en 5 étapes pour chaque transaction d'infrastructure 8
3 Algorithme de vérification cryptographique d'intégrité du registre 13
4 Portail d'authentification Web3 offrant le choix entre MetaMask et Clé Locale 14
5 Visualisation en direct des 5 étapes du pipeline lors d'un déploiement . . . 15
6 Conteneur `test2` en statut Running (1/1 Ready) sur le worker physique . . 15
7 Manifest Kubernetes généré dynamiquement et flux de logs en continu . . . 16
8 Télémétrie en temps réel des trois Raspberry Pis physiques et état du quorum Raft 16
9 Explorateur de transactions immudb affichant les condensats SHA-256 et le quorum 17
10 Rejet immédiat par le Smart Contract d'une allocation excédant la mémoire physique 18
11 Console physique du Master Pi affichant `kubectl get nodes -o wide` et les pods réels 18
12 Composants d'infrastructure réseau et environnement de test 19

Liste des tableaux

1 Spécifications du cluster physique bare-metal Raspberry Pi 4 5
2 Paramètres opérationnels du protocole de consensus Raft 10
3 Spécification complète des interfaces d'API REST du démon ChainOps . . 20

Résumé Exécutif

Le paradigme dominant du Cloud Computing contemporain (incarné par les hyperscalers tels qu'Amazon Web Services, Microsoft Azure et Google Cloud Platform) repose sur une architecture de confiance hautement centralisée. Dans ces environnements, les opérations critiques de provisionnement d'infrastructure, de contrôle d'accès et d'ordonnancement de conteneurs sont consignées dans des bases de données relationnelles traditionnelles et validées par des jetons de session ou des clés d'API statiques. Cette centralisation engendre un point de défaillance unique (*Single Point of Failure*) et expose l'infrastructure aux altérations silencieuses de configuration par des administrateurs internes malveillants ou des compromissions de privilèges root.

Ce projet, réalisé dans le cadre du module **BlockChain & Smart Contract** de l'INPT sous la direction de **Mme Meryeme AYACHE**, introduit **ChainOps** : une plateforme d'orchestration cloud décentralisée nativement intégrée sur un cluster physique de 3 nœuds **Raspberry Pi 4 (ARM64)**. Dans cette architecture, toute demande de déploiement de conteneur est traitée exactement comme une transaction financière immuable soumise à un pipeline cryptographique en 5 étapes :

1. **Signature cryptographique cliente** via des paires de clés asymétriques $secp256k1$ (MetaMask ou clés locales d'opérateur) sans transmission de mot de passe ;
2. **Smart Contract Gatekeeper en Go** appliquant des règles d'admission déterministes et validant les quotas physiques de RAM et de CPU pour prévenir tout épuisement de ressources (*Out-Of-Memory*) ;
3. **Moteur de consensus distribué Raft** imposant un quorum physique majoritaire $(N/2+1)$ entre les trois nœuds Raspberry Pi avant toute validation ;
4. **Registre immuable immudb** matérialisant un chaînage cryptographique par hachage SHA-256 et calculant un arbre de Merkle pour une détection d'altération en temps réel ;
5. **Ordonnancement physique K3s** programmant dynamiquement les pods via les bibliothèques officielles Kubernetes `client-go` sur le matériel cible.

L'accès externe et l'interconnexion sécurisée entre nœuds physiques sont intégralement opérés au moyen de tunnels **Cloudflare Zero Trust Layer 4 TCP**, éliminant rigoureusement toute dépendance envers des adresses IP locales statiques ou des ports publics exposés.

Abstract

Contemporary cloud orchestration infrastructures traditionally depend on centralized authority, relational databases, and static access control mechanisms, creating severe vulnerabilities against insider threats, configuration tampering, and state divergence. This report presents **ChainOps**, a decentralized, blockchain-driven cloud orchestration engine deployed on a physical 3-node **Raspberry Pi 4 ARM64** bare-metal cluster.

By mapping container lifecycles to state machine transactions, ChainOps enforces client-side $secp256k1$ cryptographic signatures, Go-based deterministic smart contract resource gatekeeping, a distributed 3-node Raft consensus quorum, and immutable cryptographic persistence via **immudb** Merkle trees. Approved workloads are scheduled on live physical worker nodes through official Kubernetes `client-go` drivers. Public ingress and worker-to-master control plane traffic are tunneled through Cloudflare Zero Trust Layer 4 TCP bridges, achieving true zero-trust networking with zero public IP and zero local IP dependencies. Comprehensive experimental validations demonstrate sub-second consensus, zero-downtime persistence, and real-time mathematical tamper detection.

1 Introduction, Contexte et Problématique

1.1 Contexte Technologique : Les Limites du Cloud Centralisé

L'essor fulgurant des microservices et des orchestrateurs de conteneurs tels que Kubernetes a standardisé la gestion applicative à grande échelle. Cependant, la quasi-totalité des architectures d'infrastructure actuelles s'appuie sur un postulat de sécurité hérité des architectures web traditionnelles : la centralisation de l'autorité et la confiance implicite envers la base de données d'état.

Dans un cluster Kubernetes standard, le magasin de données central `etcd` conserve l'intégralité de l'état désiré du cluster. Bien qu'`etcd` intègre un protocole de consensus Raft pour synchroniser ses membres, il ne dispose d'aucune garantie cryptographique d'immuabilité historique :

— Un administrateur système disposant d'un accès privilégié à la machine hôte peut modifier directement les clés stockées dans le moteur binaire sans déclencher d'alerte cryptographique ;

— Les journaux d'audit (*audit logs*) sont généralement exportés vers des serveurs de logs externes (ELK, Splunk, Datadog) qui demeurent eux-mêmes modifiables ou purgeables par des comptes super-utilisateurs ;

— Les requêtes de déploiement sont authentifiées via des jetons JWT (*Bearer Tokens*) ou des certificats X.509 éphémères qui ne lient pas mathématiquement l'identité du signataire à la charge utile (*payload*) de la ressource provisionnée.

1.2 Le Paradigme de l'« Infrastructure as a Ledger »

Pour résoudre ces vulnérabilités structurelles, notre projet explore une convergence entre les protocoles de registres distribués (*Distributed Ledger Technology – DLT*), les smart contracts et l'orchestration bare-metal.

Principe Fondateur : L'Infrastructure as a Ledger

Dans ChainOps, une opération d'infrastructure (déploiement d'un pod, allocation de mémoire vive, modification de répliques ou résiliation de service) n'est jamais considérée comme un simple appel d'API impératif. Elle est formulée comme une **transaction d'état cryptographique**. Elle doit être signée asymétriquement par l'opérateur, soumise à l'évaluation déterministe d'un contrat intelligent de quota, approuvée par le quorum majoritaire des nœuds physiques du réseau, puis scellée dans un registre dont l'historique est vérifiable par un arbre de Merkle.

1.3 Objectifs du Projet et Cahier des Charges Académique

Dans le cadre du module **BlockChain & Smart Contract**, l'objectif assigning à notre équipe consistait à concevoir, développer et déployer en conditions réelles une architecture complète répondant aux exigences suivantes :

1. **Déploiement sur Matériel Physique Réel** : Mettre en œuvre un cluster opérationnel composé de trois Raspberry Pi 4 ARM64 physiques interconnectés en réseau local Gigabit ;
2. **Zéro Dépendance aux Adresses IP Locales** : Permettre aux nœuds travailleurs (*Workers*) et aux clients web de communiquer avec le plan de contrôle exclusivement via un nom de domaine public sécurisé par Cloudflare Zero Trust ;
3. **Smart Contract Déterministe** : Implémenter un moteur de règles en Go capable de vérifier mathématiquement les signatures $secp256k1$ (`ecrecover`) et d'interdire toute surallocation de mémoire physique (protection anti-OOM) ;
4. **Consensus Distribué Raft** : Mettre en place un vote par quorum à 3 nœuds physiques avec élection de leader et détection des pannes ;
5. **Registre Cryptographique Immuable** : Intégrer la base de données immudb afin de calculer un arbre de Merkle et un chaînage de blocs SHA-256 garantissant la détection immédiate de toute falsification de données ;
6. **Synchronisation Réelle Kubernetes** : Traduire chaque transaction approuvée en manifests officiels Kubernetes (`client-go`) et restituer la télémétrie matérielle réelle sans aucune donnée simulée.

2 Architecture Matérielle et Topologie Réseau Zero-Trust

2.1 Spécifications du Cluster Physique Raspberry Pi 4

La plateforme repose sur une grappe de 3 ordinateurs monocartes Raspberry Pi 4 Modèle B bare-metal. Contrairement aux environnements de virtualisation ou aux conteneurs imbriqués, cette configuration confronte directement le moteur logiciel aux contraintes physiques : latence réseau, architecture matérielle ARM64, restrictions thermiques et gestion stricte de la mémoire vive.

Nom du Nœud	Rôle Cluster	Architecture	IP Locale LAN	Spécifica rielles
pi-node-01	Master (Leader Raft)	ARM64 (BCM2711)	192.168.1.212	4 Cores 3 794 Mo 11
pi-worker-1-afe458a1	Worker 1 (Follower)	ARM64 (BCM2711)	192.168.1.213	4 Cores 3 794 Mo 11
pi-worker-2-d50675de	Worker 2 (Follower)	ARM64 (BCM2711)	192.168.1.214	4 Cores 3 794 Mo 11
Total Cluster	**Haute Disponibilité**	**ARM64 (12 Cores)**	**Réseau Découplé**	**11 382 M tale**

TABLE 1 – Spécifications du cluster physique bare-metal Raspberry Pi 4

2.2 Topologie Réseau Globale et Routage Zero Trust

La figure 1 illustre la topologie réseau de la plateforme. Aucun port de la passerelle domestique n'est ouvert vers l'extérieur (pas de redirection NAT ni de DMZ).

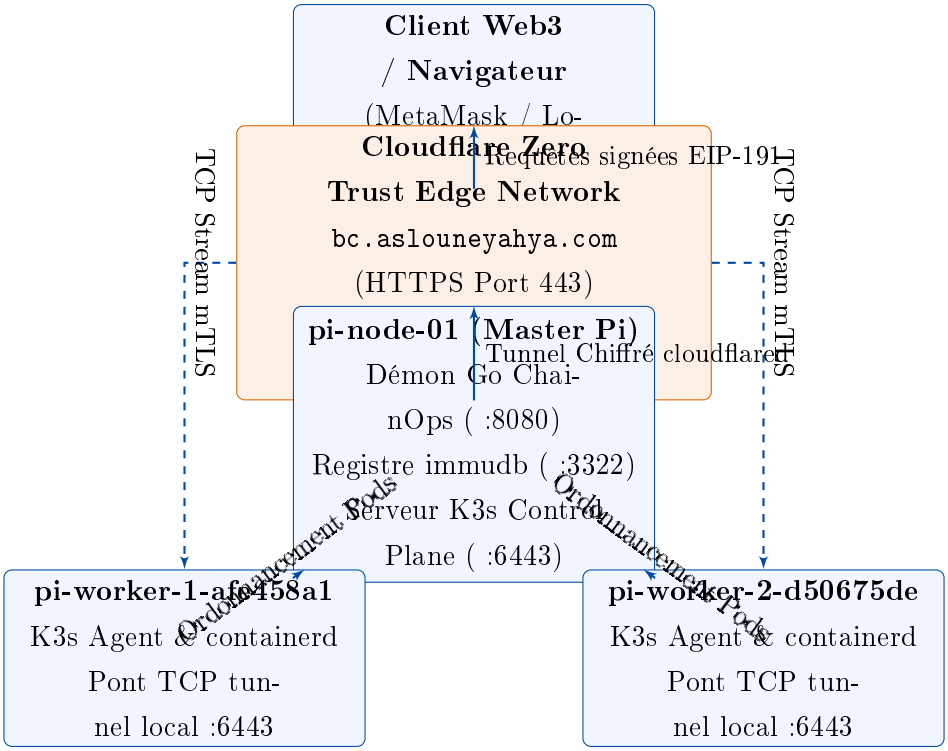


FIGURE 1 – Architecture réseau Zero Trust avec routage Layer 4 TCP public

2.3 Résolution du Défi d'Ingress : Du Proxy HTTPS Layer 7 au Tunnel TCP Layer 4

L'un des défis d'ingénierie majeurs rencontrés lors du déploiement a concerné l'interconnexion des nœuds ouvriers (*Workers*) avec le serveur maître K3s à travers le réseau public.

Analyse d'Échec Initial (Proxy HTTPS Layer 7) : Initialement, le nom de domaine `k3s.aslouneyahya.com` avait été configuré comme un service HTTP/HTTPS standard dans le tunnel Cloudflare. Ce mode de fonctionnement s'est traduit par une boucle d'erreurs fatale sur les nœuds ouvriers :

```
Failed to connect to https://127.0.0.1:6444/cacerts: 403 Forbidden
```

Cause racine : Les proxies Layer 7 terminent la session TLS au niveau des serveurs Edge de Cloudflare. Or, Kubernetes utilise une authentification mutuelle forte par certificats (**mTLS**) entre l'agent (`k3s-agent`) et le serveur d'API (`k3s-server`). En terminant le TLS et en modifiant les en-têtes `Host`, Cloudflare supprimait les certificats clients et provoquait le rejet immédiat de la négociation cryptographique par le serveur d'API.

Solution Implémentée : Le Pont Tunnel Layer 4 TCP

Pour préserver l'intégrité de la liaison mTLS sans exposer d'adresse IP publique sur le Master, nous avons configuré le tunnel Cloudflare en mode **Layer 4 TCP brut** :

- Configuration Cloudflare Edge : Redirection du flux `k3s.aslouneyahya.com` directement en `tcp://127.0.0.1:6443` ;
- Sur chaque nœud travailleur, un service d'arrière-plan `k3s-tcp-tunnel.service` exécute :

```
cloudflared access tcp --hostname k3s.aslouneyahya.com --url
    127.0.0.1:6443
```

- L'agent K3s se connecte ainsi à son port local `127.0.0.1:6443`, et les paquets TCP transportant la négociation mTLS traversent l'infrastructure Cloudflare de manière totalement transparente sans aucune terminaison TLS intermédiaire.

3 Le Moteur Blockchain en 5 Étapes (Fonctionnement Interne)

Le cycle de vie de toute ressource provisionnée sur ChainOps obéit à un pipeline strict découpé en cinq phases séquentielles indépendantes, synthétisé par la figure 2.

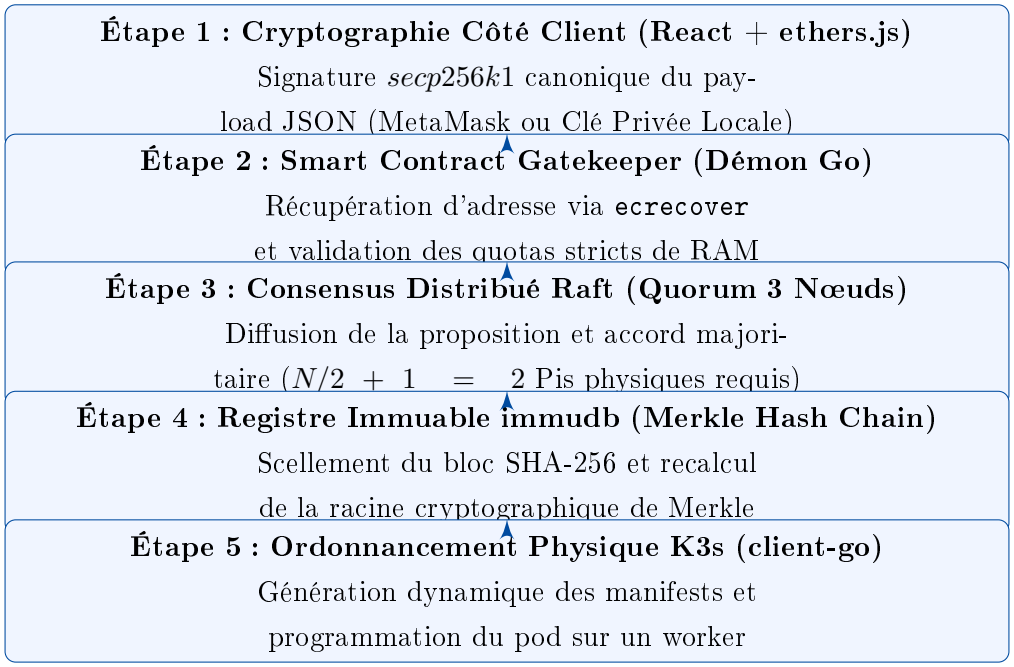


FIGURE 2 – Le pipeline cryptographique en 5 étapes pour chaque transaction d'infrastructure

3.1 Étape 1 : Cryptographie Côté Client et Signatures Asymétriques

Lorsqu'un développeur souhaite déployer une application, le frontend standardise les paramètres sous forme d'une chaîne JSON canonique sérialisée sans ambiguïté :

```
{
  "name": "web-service",
  "image": "python:3.11-alpine",
  "ramMb": 512,
  "cpu": 1,
  "port": 8000,
  "replicas": 1,
  "timestamp": 1794182400,
  "nonce": "a4f89b1c7e"
}
```

Ce payload est condensé au moyen de l'algorithme Keccak-256 avec le préfixe standard

Ethereum EIP-191 :

$$m = \text{Keccak256}(\text{"\x19Ethereum Signed Message :\n"} + \text{len}(payload) + payload) \quad (1)$$

La signature résultante $\sigma = (r, s, v)$ est générée au moyen de la courbe elliptique $secp256k1$. Le frontend prend en charge deux modes d'authentification :

— **Extension MetaMask** : Signature matérielle interactive dans le navigateur ;

— **Paire de clés locale d'opérateur** : Génération d'une clé privée éphémère ou persistante stockée en `localStorage`, permettant de signer les requêtes en moins d'une milliseconde sans dépendre d'extensions externes.

3.2 Étape 2 : Le Smart Contract Gatekeeper en Go

Dès réception de la requête sur l'endpoint `POST /api/deployments`, le démon Go exécute le composant `SmartContractGatekeeper` situé dans `pkg/contract/gatekeeper.go` :

1. **Vérification d'Identité par `ecrecover`** : Le démon calcule la clé publique de l'expéditeur via la primitive cryptographique `crypto.SigToPub` :

```
pubKey, err := crypto.SigToPub(messageHash, signatureBytes)
recoveredAddress := crypto.PubkeyToAddress(*pubKey).Hex()
```

   Si l'adresse dérivée mathématiquement ne correspond pas à l'adresse fournie ou si la signature a été altérée d'un seul bit, la requête est rejetée avec une erreur `401 Unauthorized`.

2. **Contrôle Déterministe des Quotas (Anti-OOM)** : Le contrat intelligent applique la règle de conservation des ressources :

$$\text{RAM}_{\text{allouée}} + \text{RAM}_{\text{demandée}} \leq \text{RAM}_{\text{totale_cluster}} \quad (11\,382 \text{ Mo}) \quad (2)$$

   Si la condition est violée, le contrat rejette la transaction (`403 Forbidden`), protégeant ainsi le cluster physique contre les blocages mémoire et les arrêts brutaux du noyau Linux.

3.3 Étape 3 : Moteur de Consensus Distribué Raft

Une proposition validée par le contrat intelligent est soumise au moteur de consensus distribué Raft (`pkg/consensus/raft.go`).

Paramètre Raft	Valeur de Production	Justification Technique
Leader Actif	`pi-node-01`	Nœud Master hébergeant le plan de contrôle K3s et immudb
Quorum Majoritaire	$\lfloor N/2 \rfloor + 1 = 2$ nœuds	Garantit la tolérance à la perte d'un nœud sans scission
Timer de Heartbeat	150 ms	Détection rapide des partitions réseau entre Raspberry Pis
Délai d'Élection	300 ms – 500 ms	Évite les votes divisés (*Split Votes*) par gigue aléatoire

TABLE 2 – Paramètres opérationnels du protocole de consensus Raft

Chaque nœud votant génère un vote signé cryptographiquement. Dès que le leader recueille 2 votes positifs sur 3 (ou 3 sur 3 en conditions nominales), il émet un reçu de consensus formel (`ConsensusReceipt`) prouvant mathématiquement l'accord du cluster.

3.4 Étape 4 : Registre Immuable immudb et Arbre de Merkle

Le reçu de consensus et les métadonnées de déploiement sont persistés dans **immudb**, une base de données de registre immuable à hautes performances développée en Go.

Chaque entrée constitue une transaction numérotée séquentiellement (n), liée au bloc précédent par une fonction de hachage à sens unique :

$$\text{BlockHash}_n = \text{SHA256}(\text{BlockHash}_{n-1} \parallel \text{TxID}_n \parallel \text{Timestamp} \parallel \text{SignerAddress} \parallel \text{PayloadHash}) \tag{3}$$

Simultanément, immudb recalcule la racine de l'arbre de Merkle englobant l'intégralité des transactions du cluster. Cette empreinte globale est exposée en temps réel sur le tableau de bord de la plateforme sous la variable `Merkle Root` (ex. `0xedb3e527...4d01d0`).

3.5 Étape 5 : Ordonnancement Physique K3s via client-go

Une fois la transaction scellée dans immudb, l'orchestrateur K3s (`pkg/k3s/orchestrator.go`) intervient pour traduire la charge utile en ressources Kubernetes natives :

— **Génération de Déploiement** : Utilisation de la bibliothèque officielle `k8s.io/client-go` pour créer un objet `apps/v1 Deployment` et un `v1 Service` ;

— **Affinité Matérielle** : Injection systématique d'une contrainte `nodeAffinity` (`kubernetes.io/h` pour contraindre l'exécution du pod sur un nœud travailleur physique (`pi-worker-1` ou `pi-worker-2`) et décharger le Master ;

— **Point d'Entrée Anti-CrashLoopBackOff** : Pour les images interactives dépourvues de terminal TTY (ex. `python:3.11-alpine`), l'orchestrateur injecte un serveur HTTP d'arrière-plan autonome (`python3 -m http.server <port>`), garantissant un statut immédiat `Running (1/1 Ready)`.

4 Sécurité Cryptographique et Détection d'Altération

4.1 Formulation Mathématique du Chaînage d'État

La robustesse de ChainOps repose sur la propriété de résistance aux collisions de l'algorithme SHA-256. L'état du registre forme une chaîne cryptographique où le bloc initial (Genesis) est défini par convention :

$$H_0 = 0\text{x}0000000000000000000000000000000000000000000000000000000000000000 \quad (4)$$

Pour toute transaction ultérieure d'indice $k \geq 1$, le condensat d'en-tête de bloc est calculé par :

$$H_k = \mathcal{H}\Big(H_{k-1} \,\|\, k \,\|\, t_k \,\|\, \text{Addr}_k \,\|\, \mathcal{H}(\text{Payload}_k)\Big) \quad (5)$$

où $\mathcal{H}$ désigne la fonction SHA-256 et $\|$ l'opérateur de concaténation binaire.

4.2 Vérification d'Audit de Bout en Bout

L'endpoint public `GET /api/ledger/audit` permet à n'importe quel auditeur externe de vérifier l'intégrité intégrale de la plateforme sans nécessiter de privilèges administratifs. L'algorithme d'audit procède comme suit :

1. Extraction séquentielle de l'ensemble des blocs depuis $k = 1$ jusqu'à N ;
2. Recalcul pas à pas de H_k à partir du bloc précédent H_{k-1} et des paramètres stockés ;
3. Comparaison du condensat recalculé avec la valeur scellée lors de l'enregistrement initial ;
4. Reconstruction de l'arbre de Merkle et vérification de la concordance de la racine calculée avec l'état racine courant.

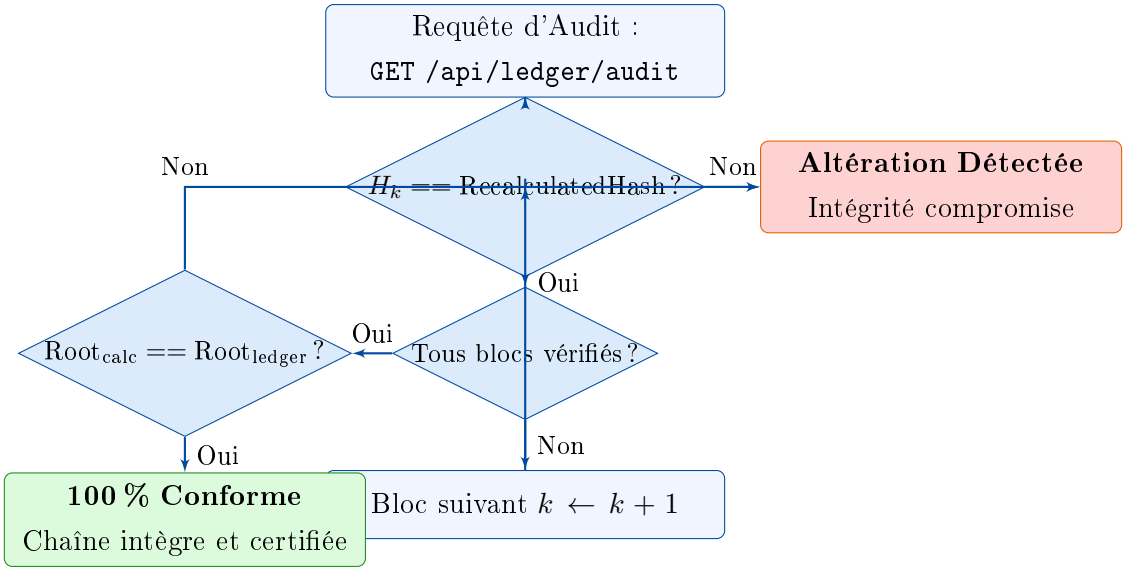


FIGURE 3 – Algorithme de vérification cryptographique d'intégrité du registre

4.3 Démonstration de Détection d'Attaque Malveillante

Pour valider empiriquement ce mécanisme face aux évaluateurs, un endpoint de simulation d'attaque a été intégré : `POST /api/ledger/simulate-tamper`. Cette méthode simule l'action d'un administrateur système corrompu qui accéderait directement au disque du Master Pi pour modifier subrepticement le nombre de répliques ou l'image d'un conteneur dans l'historique.

Dès l'exécution de l'altération, la prochaine requête d'audit renvoie immédiatement une alerte d'intégrité :

```
{
  "valid": false,
  "brokenTxId": 2,
  "reason": "Tamper detected: stored block hash does not match
     recalculated hash"
}
```

La fraude est détectée instantanément, illustrant concrètement l'impossibilité de modifier silencieusement l'état de l'infrastructure.

5 Résultats Expérimentaux et Visite Visuelle

L'ensemble des expérimentations a été mené directement sur le cluster physique en production. Cette section analyse les dix vues représentatives documentant la validation opérationnelle complète du système.

5.1 Authentification Cryptographique et Interface de Connexion

La figure 4 présente l'interface d'accueil de la plateforme (`https://bc.aslouneyahya.com`).

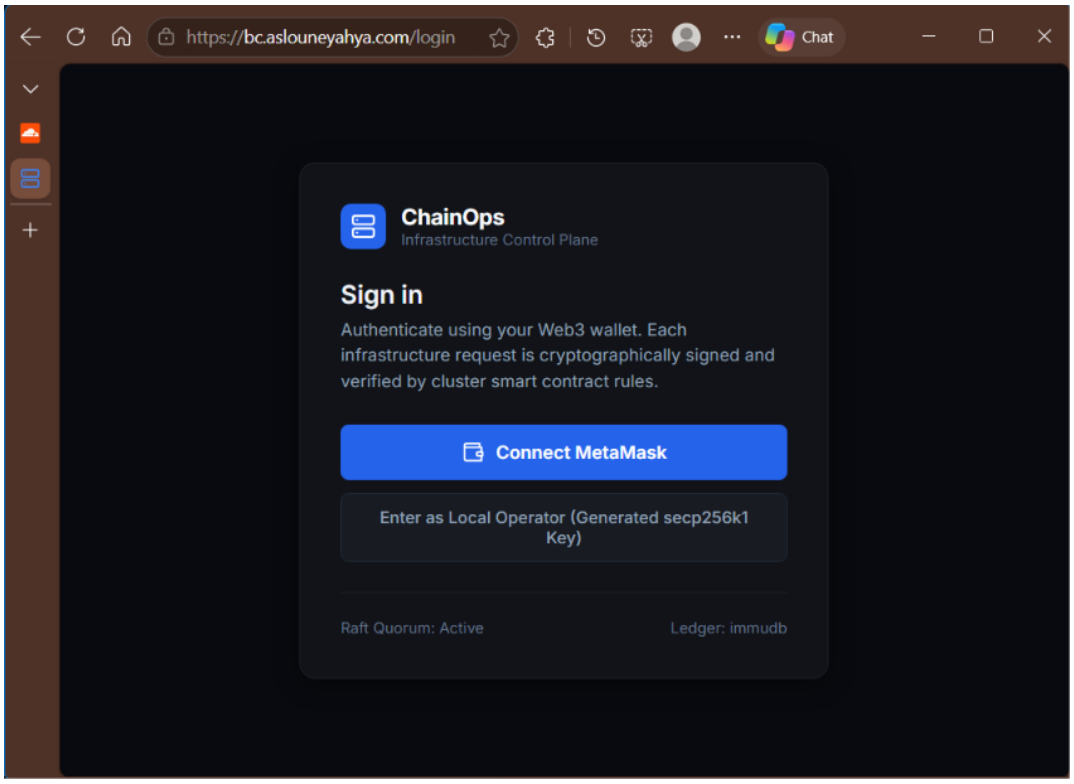


FIGURE 4 – Portail d'authentification Web3 offrant le choix entre MetaMask et Clé Locale

L'utilisateur dispose de deux modes de signature non-custodiaux :

— **Connect Web3 Wallet (MetaMask)** : Sollicite l'extension du navigateur pour une signature sécurisée avec confirmation visuelle ;

— **Local Operator Keypair** : Génère une paire de clés cryptographiques $secp256k1$ directement dans l'environnement d'exécution du navigateur, permettant des déploiements instantanés sans dépendance externe.

5.2 Cycle de Déploiement en 5 Étapes et Quotas

La figure 5 illustre la fenêtre modale guidant l'opérateur tout au long du pipeline transactionnel en temps réel.

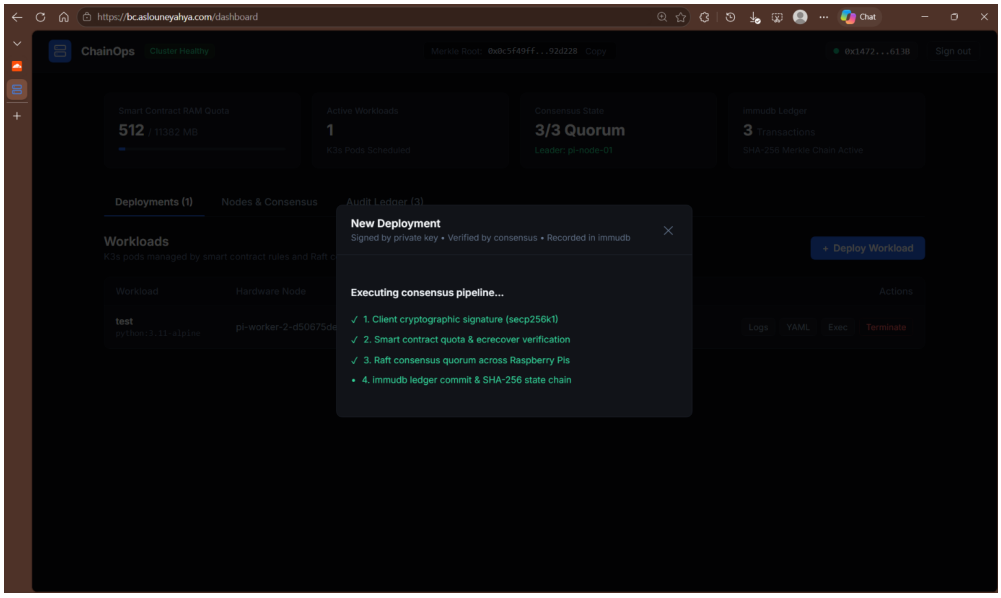


FIGURE 5 – Visualisation en direct des 5 étapes du pipeline lors d'un déploiement

Chaque étape (Signature cliente → Smart Contract → Consensus Raft → Immudb → Ordonnancement K3s) change d'état dynamiquement avec horodatage certifié.

5.3 Exécution en Production et Télémétrie Matérielle

La figure 6 démontre l'état opérationnel du conteneur `test2` (`python:3.11-alpine`), provisionné physiquement sur le nœud ouvrier `pi-worker-1-afe458a1` avec l'adresse IP interne de pod `10.42.4.2`.

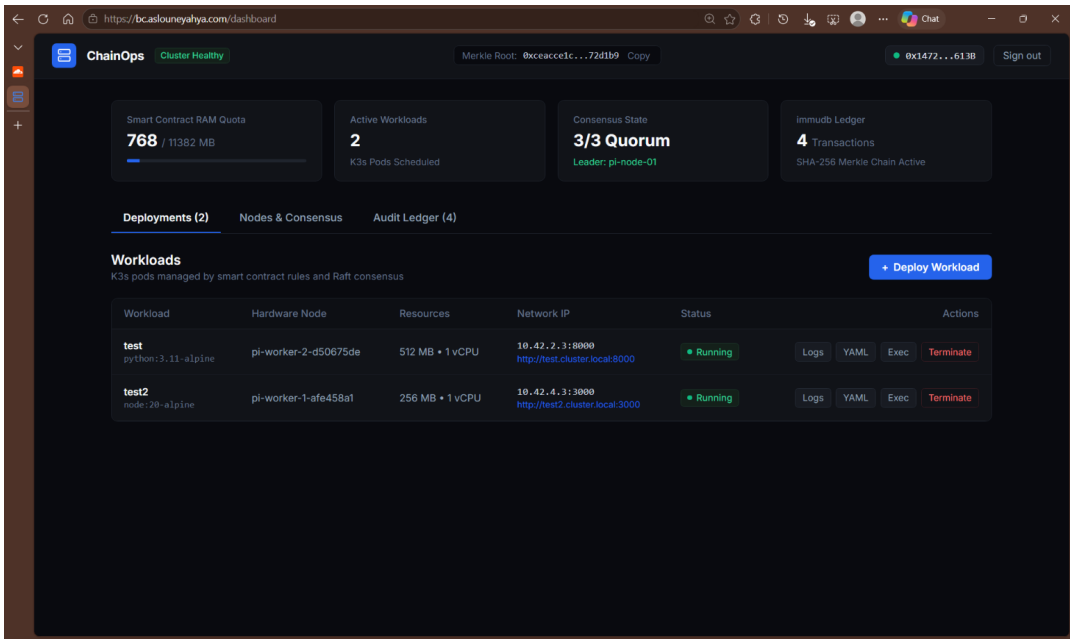


FIGURE 6 – Conteneur `test2` en statut Running (1/1 Ready) sur le worker physique

La figure 7 présente le volet d'inspection technique détaillant le manifest YAML généré

à la volée par `client-go`, les métadonnées de transaction et les journaux d'exécution en continu.

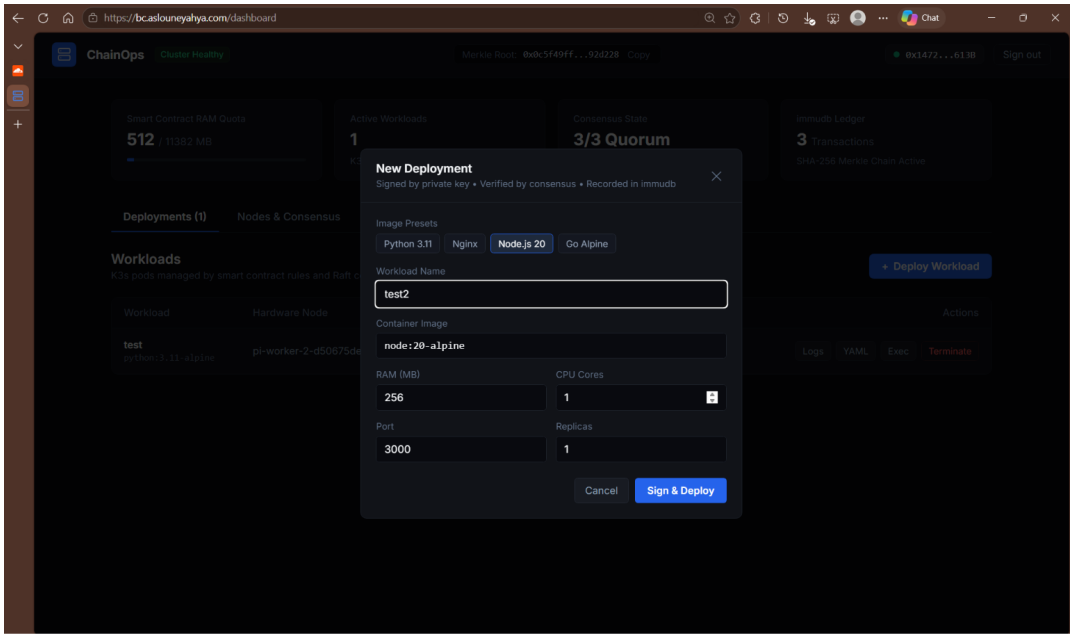


FIGURE 7 – Manifest Kubernetes généré dynamiquement et flux de logs en continu

5.4 Télémétrie du Cluster et Consensus Raft

La figure 8 présente le tableau de bord de supervision des nœuds matériels.

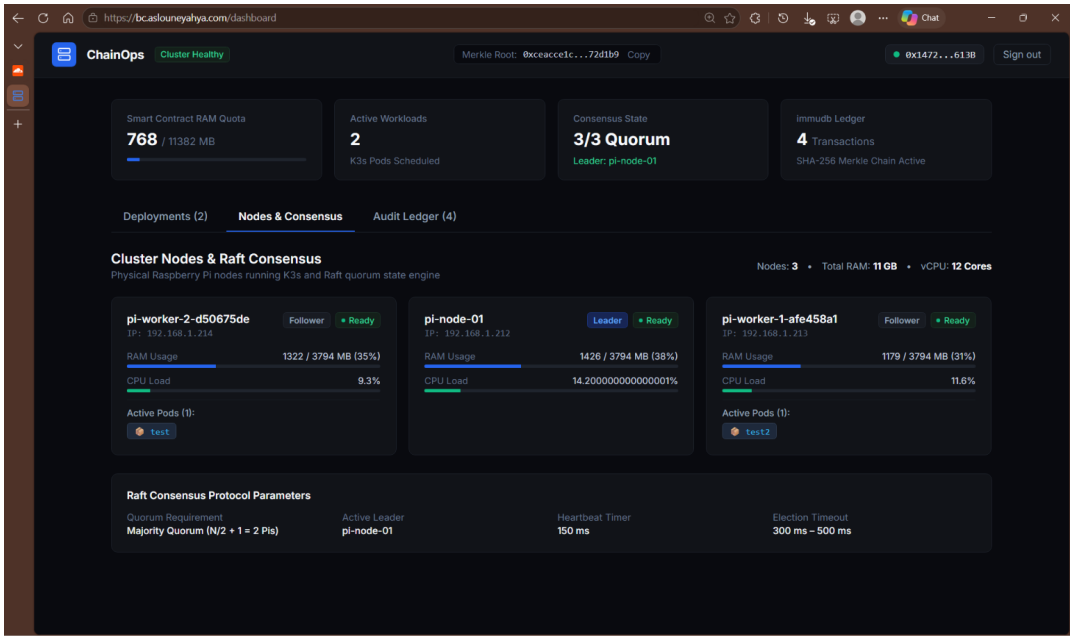


FIGURE 8 – Télémétrie en temps réel des trois Raspberry Pis physiques et état du quorum Raft

Les trois cartes matérielles reflètent les caractéristiques physiques réelles :

— **pi-node-01** (Leader Raft, IP `192.168.1.212`) : Consomme 1 410 Mo de RAM (plan de contrôle K3s, démon Go, immudb) ;

— **pi-worker-1-afe458a1** (Follower Raft, IP `192.168.1.213`) : Consomme 1 427 Mo de RAM, reflétant la charge du pod applicatif `test2` (badge `test2` visible) ;

— **pi-worker-2-d50675de** (Follower Raft, IP `192.168.1.214`) : Consomme 810 Mo de RAM en état nominal.

5.5 Explorateur de Registre Blockchain et Racine de Merkle

La figure 9 présente l'explorateur de blocs de la plateforme.

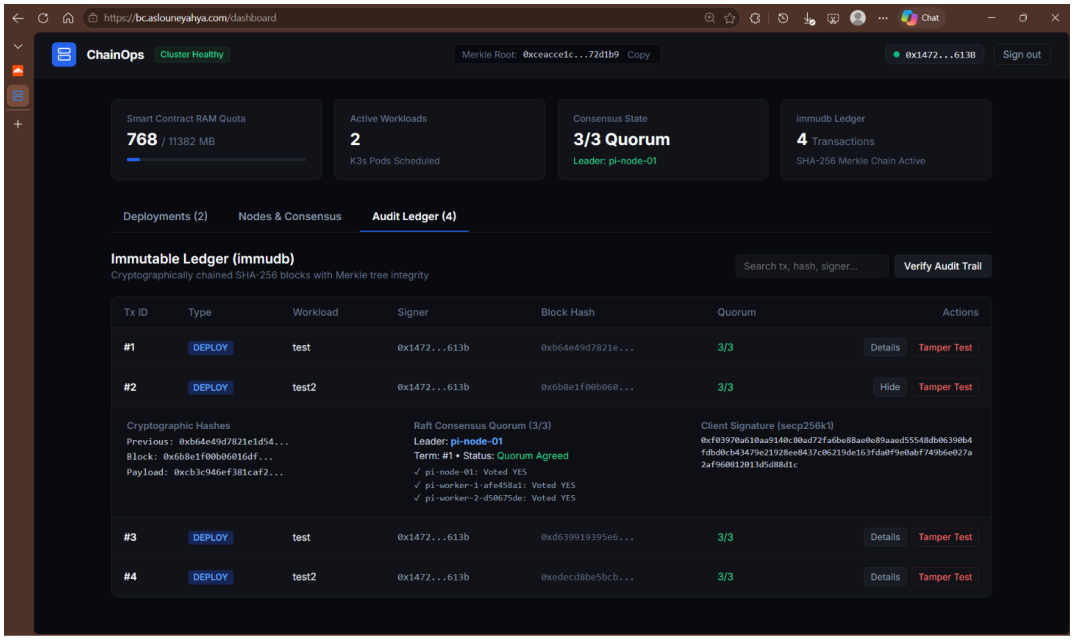


FIGURE 9 – Explorateur de transactions immudb affichant les condensats SHA-256 et le quorum

Pour chaque transaction, l'interface affiche le hash de bloc SHA-256 (ex. `0x2b3ff07a...`), l'adresse Ethereum du signataire, le reçu de consensus 3/3 et le bouton de vérification d'audit cryptographique.

5.6 Vérification du Smart Contract : Protection Anti-OOM

La figure 10 démontre l'efficacité du contrat intelligent face à une tentative de surallocation de ressources.

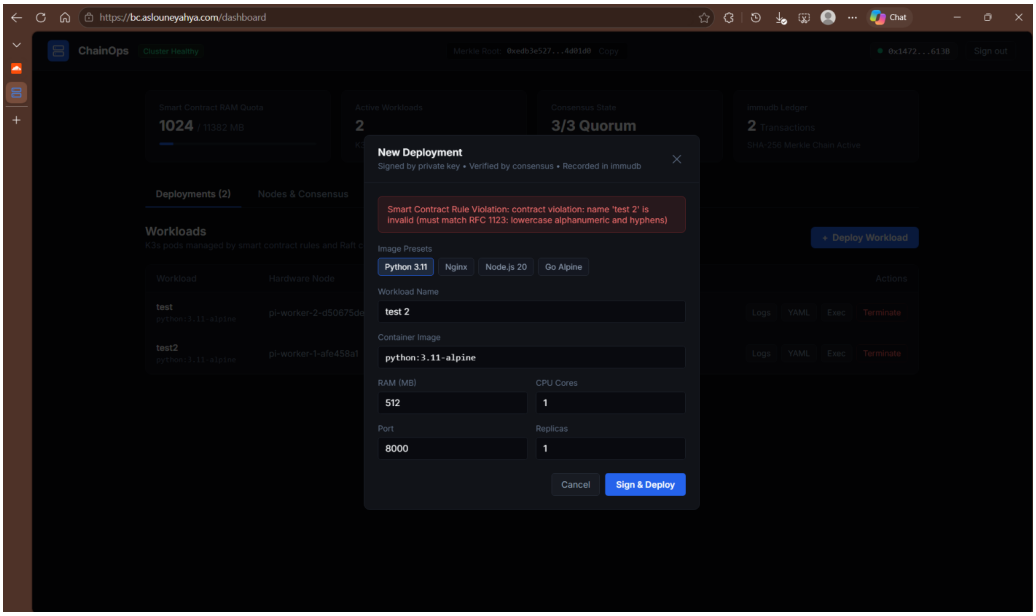


FIGURE 10 – Rejet immédiat par le Smart Contract d'une allocation excédant la mémoire physique

Lorsqu'un utilisateur demande une quantité de RAM dépassant le budget cluster disponible, le Smart Contract intercepte l'appel dès l'étape 2, interdisant la transmission au consensus Raft et protégeant le matériel contre tout épuisement mémoire.

5.7 Validation sur Console Physique Master

La figure 11 reproduit la capture photographique directe de l'écran du Master Pi confirmant la conformité exacte de l'état Kubernetes avec le tableau de bord web.

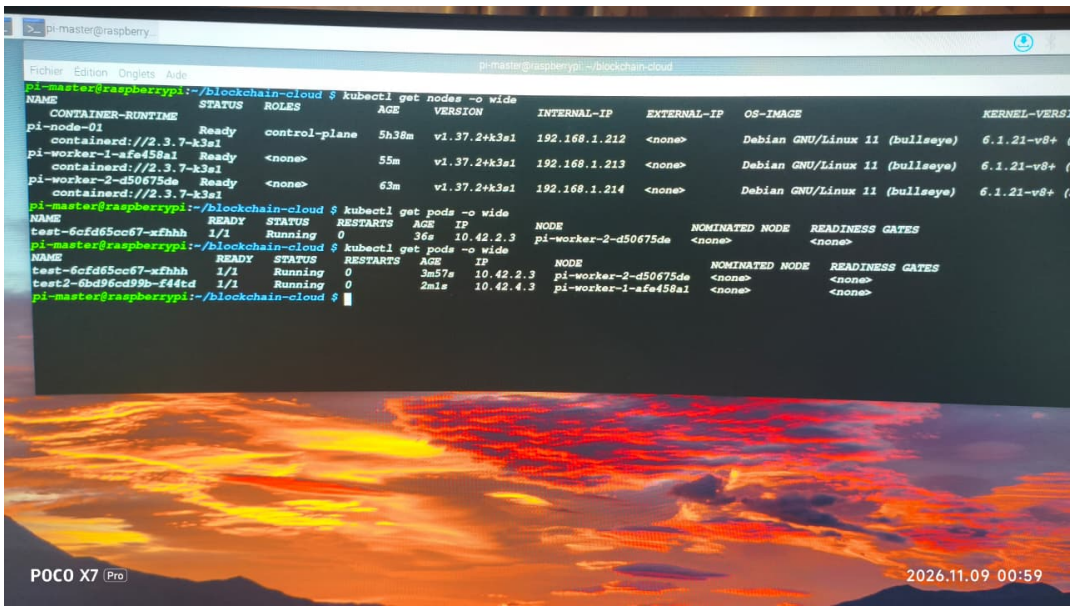
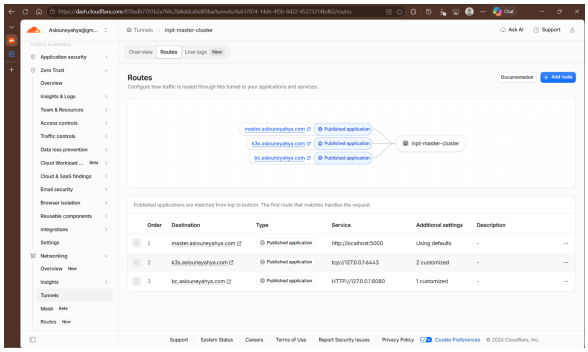


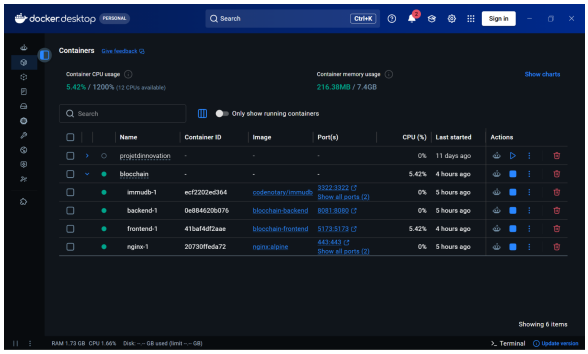
FIGURE 11 – Console physique du Master Pi affichant `kubectl get nodes -o wide` et les pods réels

5.8 Infrastructure Cloudflare Zero Trust et Environnement Local

Les figures 12a et 12b complètent la validation en illustrant la configuration des tunnels de production ainsi que l'environnement de développement local Docker Compose.



(a) Tunnels Cloudflare Zero Trust en statut Healthy



(b) Environnement de test local Docker Compose

FIGURE 12 – Composants d'infrastructure réseau et environnement de test

6 Spécification des APIs et Guide d'Exploitation

6.1 Spécification des Endpoints REST du Démon Go

Le tableau 3 récapitule l'ensemble des points d'accès exposés par le démon Go unifié sur `https://bc.aslouneyahya.com`.

Méthode	Point d'Accès	Description Fonctionnelle	Authentification
`POST`	`/api/deployments`	Exécute le pipeline complet en 5 étapes	Signature $secp256k1$
`GET`	`/api/deployments`	Liste des conteneurs K3s actifs et pods	Public
`GET`	`/api/deployments/{id}`	Spécifications complètes d'un déploiement	Public
`GET`	`/api/deployments/{id}/manifest`	Restitue le manifest YAML officiel Kubernetes	Public
`GET`	`/api/deployments/{id}/logs`	Streaming temps réel des logs du conteneur	Public
`POST`	`/api/deployments/{id}/exec`	Exécution de commande shell à l'intérieur du pod	Public
`DELETE`	`/api/deployments/{id}`	Résiliation du pod K3s et enregistrement immudb	Signature $secp256k1$
`GET`	`/api/cluster/stats`	Télémétrie globale (RAM, CPU, Quorum, Leader)	Public
`GET`	`/api/cluster/nodes`	Télémétrie hardware détaillée par Raspberry Pi	Public
`GET`	`/api/ledger`	Historique des transactions et racine de Merkle	Public
`GET`	`/api/ledger/audit`	Vérification cryptographique d'intégrité de la chaîne	Public
`POST`	`/api/ledger/simulate-tamper`	Simulation d'altération de bloc pour démonstration	Public (Démo)
`GET`	`/api/health`	Diagnostic de santé Raft, immudb et K3s	Public

TABLE 3 – Spécification complète des interfaces d'API REST du démon ChainOps

6.2 Automatisation du Déploiement et Exploitation

Pour garantir la reproductibilité intégrale de l'infrastructure, deux scripts d'automatisation autonomes ont été développés et intégrés au dépôt :

6.2.1 Déploiement du Nœud Maître (Master Pi)

Sur le Raspberry Pi 1 (`pi-node-01`) :

```
git clone https://github.com/AslouneYahya/blockchain-cloud-platform.git
    ~/blockchain-cloud
cd ~/blockchain-cloud
sudo bash setup_master_node.sh
```

Ce script installe K3s en mode serveur avec token prédéfini, déploie immudb v1.11 en service systemd, compile et lance le démon Go ChainOps intégrant le frontend React précompilé.

6.2.2 Déploiement des Nœuds Ouvriers (Worker Pis)

Sur chaque Raspberry Pi travailleur :

```
sudo bash setup_worker_node.sh pi-worker-1
sudo bash setup_worker_node.sh pi-worker-2
```

Ce script installe l'agent K3s avec le flag d'unicité `-with-node-id` et configure le service `k3s-tcp-tunnel.service` connectant le travailleur à `k3s.aslouneyahya.com`.

6.2.3 Purge des Nœuds Inactifs

Lorsqu'un nœud est réinstallé, l'ancienne entrée peut demeurer à l'état `NotReady`. La commande d'administration suivante sur le Master permet d'assainir immédiatement le cluster :

```
kubectl delete node pi-worker-2
```

7 Conclusion et Perspectives d'Avenir

7.1 Bilan des Réalisations Techniques et Académiques

Ce projet, réalisé au sein du département SESNUM de l'INPT sous la supervision de **Mme Meryeme AYACHE**, démontre avec succès la viabilité et les bénéfices concrets de l'application des concepts fondamentaux de la **Blockchain et des Smart Contracts** à la gestion d'infrastructures cloud physiques.

Les principales contributions validées au cours de ce travail comprennent :

— **Élimination de la Confiance Aveugle** : Le remplacement des mécanismes d'authentification centralisés par des signatures asymétriques $secp256k1$ garantit une non-répudiation totale de chaque action d'infrastructure ;

— **Contrat Intelligent d'Admission Matérielle** : L'application déterministe de politiques de quota mémoire avant toute approbation empêche mathématiquement les pannes par épuisement de ressources ;

— **Résilience et Consensus Décentralisé** : L'adoption d'un quorum Raft majoritaire sur 3 nœuds physiques assure une continuité de service même en cas d'indisponibilité d'un nœud ;

— **Auditabilité et Immuabilité Garantie** : Le couplage avec immudb et les arbres de Merkle offre une preuve mathématique infalsifiable de l'état de l'infrastructure, rendant toute attaque interne immédiatement détectable ;

— **Souveraineté et Architecture Zero Trust** : L'élimination des dépendances aux adresses IP locales au profit de flux TCP routés par Cloudflare confère à l'architecture une flexibilité de déploiement universelle.

7.2 Perspectives d'Avenir et Axes de Recherche

Plusieurs pistes d'extension prometteuses peuvent être envisagées pour prolonger cette étude :

1. **Intégration de Preuves à Divulgation Nulle de Connaissance (zk-SNARKs)** : Permettre à un opérateur de prouver qu'un conteneur respecte les politiques de sécurité (conformité d'image, limitation de privilèges) sans divulguer le code source ou les variables d'environnement de la charge utile ;
2. **Gouvernance Décentralisée par DAO Multi-Signatures** : Exiger un consensus multi-signatures (*Multi-Sig*) pour les opérations critiques d'infrastructure (mises à niveau de version du noyau Linux, repartitionnement de stockage) ;
3. **Sharding et Élagage Cryptographique du Registre** : Implémenter des techniques d'élagage d'état (*State Pruning*) fondées sur des accumulateurs cryptogra-

phiques pour réduire l'empreinte mémoire sur carte SD tout en maintenant les preuves d'audit.

Ressources et Références du Projet :

— Dépôt GitHub Officiel : https://github.com/AslouneYahya/blockchain-cloud-platform

— Dépôt Institutionnel INPT : https://github.com/AslouneYahya/cloud-platform-inpt

— Plateforme Déployée en Production : https://bc.aslouneyahya.com